

Les variables de verrouillage

Access à plusieurs ressources

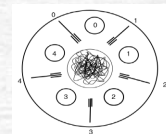
- Le problème de l'exclusion mutuelle considère le cas de plusieurs threads qui se partagent une ressource commune qui doit être manipulée de façon exclusive.
- Il est souvent nécessaire de coordonner l'accès de plusieurs threads à plusieurs ressources, chacune de ces ressources devant être utilisée de façon exclusive.
- Cette utilisation de plusieurs ressources simultanément peut poser des difficultés.
- Exemple: le problème des philosophes.
- Cinq philosophes sont assis autour d'une table. Sur la table, il y a alternativement cinq plats de spaghettis et cinq fourchettes:
 - Un philosophe passe son temps à manger et à penser.
 - Pour manger son plat de spaghettis, un philosophe a besoin de deux fourchettes, une qui se trouve à sa gauche et l'autre qui se trouve à sa droite.
 - Problème: chaque philosophe permet de se livrer à ses activités (penser et manger) sans jamais être bloqué.

39

39

Problème des philosophes

- Il est important de signaler que si tous les philosophes prennent en même temps chacun une fourchette, aucun d'entre eux ne pourra prendre l'autre fourchette (**situation d'interblocage**).
- Pour éviter cette situation, un philosophe ne prend jamais une seule fourchette. Les fourchettes sont les objets partagés.
- L'accès et l'utilisation d'une fourchette doit se faire en exclusion mutuelle.
- Ce problème peut se modéliser en utilisant un programme C avec les threads POSIX.
 - Chaque fourchette est une ressource partagée qui ne peut être utilisée que par un philosophe à la fois.
 - Chaque philosophe est modélisé par un thread.
- Écrire un programme qui permette à chaque philosophe de se livrer à ses activités (penser et manger) sans jamais être bloqué

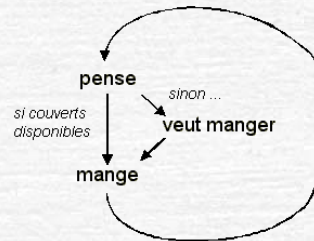
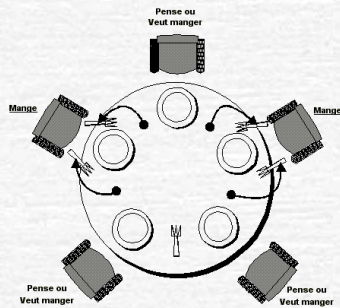


40

40

Problème des philosophes

- Quand un philosophe affamé a ses deux couverts dans les mains en même temps, il mange sans libérer ses couverts.
- Dans le cas contraire, il doit attendre que ceux-ci deviennent libres.
- Par exemple, dans notre hypothèse d'un dîner de cinq philosophes, seulement deux philosophes peuvent manger à un instant donné car les couverts ne sont pas suffisants.
- Trois philosophes n'ont la possibilité que de penser ou d'être en attente de vouloir manger.

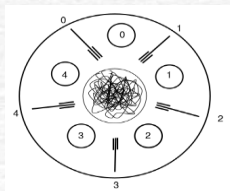


41

41

Problème des philosophes

- Scénario 1:



- Les philosophes décident de manger au même moment:
 - Commencent tous par récupérer la fourchette se trouvant à leur gauche (resp. droite).
 - Sont bloqués en essayant de récupérer la fourchette à droite (resp. gauche) (alors qu'il n'y a plus de fourchette sur la table).

```

Void philosophe()
{
    While(1)
    {
        Sleep();
        Prendre_fourchette_droite();
        Prendre_fourchette_gauche();
        Manger();
        Deposer_fourchette_droite();
        Deposer_fourchette_gauche();
    }
}
  
```

42

42

Problème des philosophes

■ Scénarios 1

```
void* philosophe (void* arg){
    int *id=(int *)arg;
    int right = *id;
    int left = (right + 1) % NUM_PHILOSOPHERS;

    while(true){
        // Philosophe pense
        printf("Philosophe [%d] pense \n", *id);
        sleep(rand() % 3); // Philosophe pense pendant un temps aléatoire

        // Tous les philosophes se synchronisent
        // avant de prendre leurs fourchettes
        pthread_barrier_wait(&barrier);

        // Philosophe essaie de prendre la fourchette gauche
        pthread_mutex_lock(&fourchettes[left]);
        printf("Philosophe [%d] a pris la fourchette gauche [%d]\n", *id, left);
        // Philosophe essaie de prendre la fourchette droite
        pthread_mutex_lock(&fourchettes[right]);
        printf("Philosophe [%d] a pris la fourchette droite [%d]\n", *id, right);

        // Philosophe mange
        mange(*id);

        // Philosophe libère la fourchette gauche
        pthread_mutex_unlock(&fourchettes[left]);
        printf("Philosophe [%d] a libéré la fourchette gauche [%d]\n", *id, left);
        // Philosophe libère la fourchette droite
        pthread_mutex_unlock(&fourchettes[right]);
        printf("Philosophe [%d] a libéré la fourchette droite [%d]\n", *id, right);
    }

    return NULL;
}
```

```
#define _GNU_SOURCE
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>
#include<stdbool.h>

#define NUM_PHILOSOPHERS 5

pthread_t phil[NUM_PHILOSOPHERS];
pthread_mutex_t fourchettes[NUM_PHILOSOPHERS];
pthread_barrier_t barrier;

void mange(int id){
    printf("Philosophe [%d] mange \n", id);
    for(int i = 0; i < rand(); i++){
        // Philosophe mange pendant un temps aléatoire
    }
}

int main(){
    int i;
    int id[NUM_PHILOSOPHERS];
    // Initialiser le générateur de nombres aléatoires
    srand(getpid());

    // Barrière pour synchroniser tous les philosophes
    pthread_barrier_init(&barrier, NULL, NUM_PHILOSOPHERS);

    // Initialiser les mutex pour les fourchettes
    for(i=0; i<NUM_PHILOSOPHERS; i++){
        pthread_mutex_init(&fourchettes[i], NULL);
    }

    for(i=0; i<NUM_PHILOSOPHERS; i++){
        id[i] = i;
        // Créer les threads pour les philosophes
        pthread_create(&phil[i], NULL, philosophe, (void*)&id[i]);
    }

    for(i=0; i<NUM_PHILOSOPHERS; i++){
        // Attendre que les threads des philosophes se terminent
        pthread_join(phil[i], NULL);
    }

    return EXIT_SUCCESS;
}
```

43

43

Problème des philosophes

■ Scénarios 1

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./philos_scenario1
Philosophe [0] pense
Philosophe [1] pense
Philosophe [2] pense
Philosophe [3] pense
Philosophe [4] pense
Philosophe [0] a pris la fourchette gauche [1]
Philosophe [2] a pris la fourchette gauche [3]
Philosophe [3] a pris la fourchette gauche [4]
Philosophe [4] a pris la fourchette gauche [0]
Philosophe [1] a pris la fourchette gauche [2]
```

44

44

Problème des philosophes

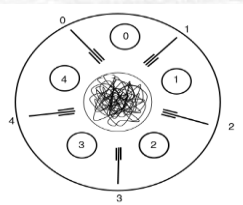
- Scenario 2: forçant les threads à s'approprier les mutex qu'ils utilisent dans le même ordre
 - Ph0: s'approprie d'abord le mutex **fourchette[0]** et ensuite le mutex **fourchette[1]**
 - Ph1: s'approprie d'abord le mutex **fourchette[1]** et ensuite le mutex **fourchette[2]**
 - Ph2: s'approprie d'abord le mutex **fourchette[2]** et ensuite le mutex **fourchette[3]**
 - Ph2: s'approprie d'abord le mutex **fourchette[3]** et ensuite le mutex **fourchette[4]**
 - Ph4: s'approprie d'abord le mutex **fourchette[4]** et ensuite le mutex **fourchette[0]**
- Avec cet ordre d'allocation des mutex, un deadlock n'est plus possible
- Il y aura toujours au moins un philosophe qui pourra s'approprier les deux baguettes dont il a besoin pour manger.
- Un deadlock ne pourrait survenir que si tous les philosophes cherchent à manger simultanément.
- Si un des philosophes ne cherche pas à manger, ses deux baguettes sont nécessairement libres et au moins un de ses voisins philosophes pourra manger.

45

45

Problème des philosophes

- Scenario 2:



```
void* philosophe (void* arg){
    int *id=(int *)arg;
    int left = *id;
    int right = (left + 1) % NUM_PHILOSOPHERS;
    while(true){
        // Philosophe pense
        printf("Philosophe [%d] pense \n", *id);
        sleep(rand() % 3); // Philosophe pense pendant un temps aléatoire

        // Fix: toujours prendre la fourchette avec le plus petit indice en premier
        if(left < right){
            // Cas normal : gauche < droite → prendre gauche d'abord
            pthread_mutex_lock(&fourchettes[left]);
            printf("Philosophe [%d] a pris la fourchette gauche [%d]\n", *id, left);
            pthread_mutex_lock(&fourchettes[right]);
            printf("Philosophe [%d] a pris la fourchette droite [%d]\n", *id, right);
        } else {
            // Cas inversé (philosophe 4) : droite < gauche → prendre droite d'abord
            pthread_mutex_lock(&fourchettes[right]);
            printf("Philosophe [%d] a pris la fourchette droite [%d]\n", *id, right);
            pthread_mutex_lock(&fourchettes[left]);
            printf("Philosophe [%d] a pris la fourchette gauche [%d]\n", *id, left);
        }

        // Philosophe mange
        mange(*id);

        // Philosophe libère les fourchettes
        pthread_mutex_unlock(&fourchettes[left]);
        printf("Philosophe [%d] a libéré la fourchette gauche [%d]\n", *id, left);
        pthread_mutex_unlock(&fourchettes[right]);
        printf("Philosophe [%d] a libéré la fourchette droite [%d]\n", *id, right);
    }
    return NULL;
}
```

46

46

Problème des philosophes

■ Scenario 2:

```

Philosophe [0] pense
Philosophe [1] pense
Philosophe [2] pense
Philosophe [4] pense
Philosophe [2] a pris la fourchette gauche [2]
Philosophe [2] a pris la fourchette droite [3]
Philosophe [2] mange
Philosophe [2] a libéré la fourchette gauche [2]
Philosophe [2] a libéré la fourchette droite [3]
Philosophe [2] pense
Philosophe [3] pense
Philosophe [3] a pris la fourchette gauche [3]
Philosophe [3] a pris la fourchette droite [4]
Philosophe [3] mange
Philosophe [3] a libéré la fourchette gauche [3]
Philosophe [3] a libéré la fourchette droite [4]
Philosophe [3] pense
Philosophe [1] a pris la fourchette gauche [1]
Philosophe [1] a pris la fourchette droite [2]
Philosophe [1] mange
Philosophe [1] a libéré la fourchette gauche [1]
Philosophe [1] a libéré la fourchette droite [2]
Philosophe [1] pense
Philosophe [1] a pris la fourchette gauche [1]
Philosophe [1] a pris la fourchette droite [2]
Philosophe [1] mange
Philosophe [1] a libéré la fourchette gauche [1]
Philosophe [1] a libéré la fourchette droite [2]

```

```

Philosophe [0] a pris la fourchette gauche [0]
Philosophe [0] a pris la fourchette droite [1]
Philosophe [0] mange
Philosophe [0] a libéré la fourchette gauche [0]
Philosophe [0] a libéré la fourchette droite [1]
Philosophe [0] pense
Philosophe [0] a pris la fourchette gauche [0]
Philosophe [0] a pris la fourchette droite [1]
Philosophe [0] mange
Philosophe [0] a libéré la fourchette gauche [0]
Philosophe [0] a libéré la fourchette droite [1]
Philosophe [0] pense
Philosophe [2] a pris la fourchette gauche [2]
Philosophe [2] a pris la fourchette droite [3]
Philosophe [2] mange
Philosophe [2] a libéré la fourchette gauche [2]
Philosophe [2] a libéré la fourchette droite [3]
Philosophe [2] pense
Philosophe [0] a pris la fourchette gauche [0]
Philosophe [0] a pris la fourchette droite [1]
Philosophe [0] mange
Philosophe [0] a libéré la fourchette gauche [0]
Philosophe [0] a libéré la fourchette droite [1]
Philosophe [0] pense

```

- Cette solution résout le problème d'interblocage. Mais, un philosophe peut mourir de faim car il ne pourra jamais obtenir les fourchettes nécessaires pour manger!!!

47

47

Plan

- La notion des threads
- Communication entre threads
- Synchronisation
 - Verou
 - Semaphore
 - Moniteur

48

48

Sémaphores

- La concurrence entre les processus peut être traité par l'emploi de sémaphores.
- Un sémaphore est un compteur entier qui désigne le nombre d'autorisations d'accès à une section critique. Il a un *nom* et une *valeur initiale*, par exemple **sémaphore S = 10**
- Un sémaphore joue donc le rôle d'un **distributeur de tickets**, qui, initialement dispose d'un certain nombre de tickets
- Les sémaphores sont manipulés au moyen des opérations **P** ou **wait** et **V** ou **signal**
 - L'opération **P(S)** (*P(uis-je) accéder à une ressource*) décrémente la valeur du sémaphore S
 - L'opération **V(S)** (*V(as-y) la ressource est disponible*) incrémente la valeur du sémaphore S
- Les sémaphores permettent de réaliser des exclusions mutuelles

```
semaphore mutex=1 ;
```

```
processus P1 :
```

```
P(mutex)
```

```
section critique de P1 ;
```

```
V(mutex) ;
```

```
processus P2 :
```

```
P(mutex)
```

```
section critique de P2 ;
```

```
V(mutex) ;
```

```
int a ;
```

```
semaphore mutex=1 ;
```

```
processus P1 :
```

```
P(mutex)
```

```
a++ ;
```

```
V(mutex) ;
```

```
processus P2 :
```

```
P(mutex)
```

```
a++ ;
```

```
V(mutex) ;
```

49

49

Sémaphores

- Principalement, un processus utilisateur du sémaphore demande un ticket en invoquant **P** :
 - si au moins un ticket est disponible, le processus appelant le prend et poursuit son exécution
 - sinon, le demandeur est enregistré dans une **file d'attente** et est bloqué dans l'attente de l'obtention d'un ticket, l'opération **P** n'est exécutable par le processus appelant que s'il existe un ticket libre
- Grace a **V**, si des processus sont en attente dans la file du sémaphore, le premier d'entre eux est débloqué et obtient son ticket.
- Le problème de l'exclusion mutuelle peut être simplement résolu à l'aide d'un sémaphore dont la valeur initiale est fixée à 1. Un tel sémaphore est couramment appelé **sémaphore d'exclusion mutuelle** (mutex).

```
semaphore mutex=1 ;
```

```
processus P1 :
```

```
P(mutex)
```

```
section critique de P1 ;
```

```
V(mutex) ;
```

```
processus P2 :
```

```
P(mutex)
```

```
section critique de P2 ;
```

```
V(mutex) ;
```

```
int a ;
```

```
semaphore mutex=1 ;
```

```
processus P1 :
```

```
P(mutex)
```

```
a++ ;
```

```
V(mutex) ;
```

```
processus P2 :
```

```
P(mutex)
```

```
a++ ;
```

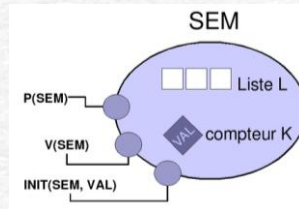
```
V(mutex) ;
```

50

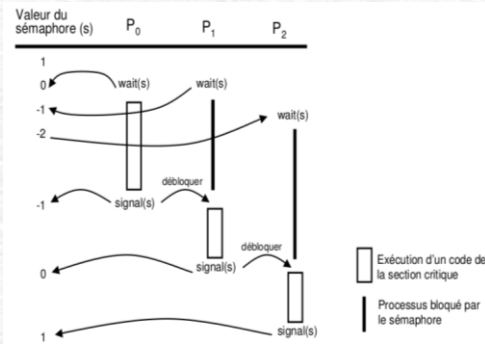
50

Sémaphores

- Un sémaphore peut donc être représenté par un compteur, et une file d'attente de processus. La file d'attente enregistre les processus demandeurs bloqués, et le compteur représente :
 - quand il est **supérieur à 0**, le nombre de tickets disponibles et donc le nombre de processus dont les demandes seront immédiatement **satisfaisantes**,
 - quand il devient **négatif**, sa valeur absolue est alors le nombre de processus **bloqués** en attente d'un ticket.



Processus 1	Processus 2
Section critique	wait(s) // attente processus 1
signal(s) // réveil processus 2	Section critique



51

51

Sémaphores Services Posix

- Les services Posix de manipulation des sémaphores se trouvent dans la librairie `<semaphore.h>`
- Le type sémaphore est désigné par le type `sem_t`.
- Initialisation d'un sémaphore:

```
int sem_init(sem_t *sem, int pshared, unsigned int value)
```

- `sem` est un pointeur sur le sémaphore à initialiser
- `value` est la valeur initiale du sémaphore
- `pshared` indique si le sémaphore est local au processus `pshared=0` ou partagé entre le père et le fils `pshared!=0`

- L'opération **P** : retourne toujours 0, suspend le thread appelant jusqu'à ce que le sémaphore pointé par `sem` ait une valeur non-nulle, il décrémente alors son compteur

```
int sem_wait(sem_t *sem)
```

52

52

Sémaphores Services Posix

- L'opération **V** : retourne 0 en cas de succès ou -1 autrement. Incrémente le compteur du sémaphore pointé par **sem**

```
int sem_post(sem_t *sem)
```

- L'opération permettant de récupérer la valeur d'un sémaphore : il retourne tjrs 0.

```
int sem_getvalue (sem_t* nom, int * sval)
```

- L'opération pour détruire un sémaphore. Retourne -1 s'il y a encore des waits

```
int sem_destroy (sem_t* nom)
```

- Les sémaphores servent donc à synchroniser les communications entre threads

53

53

Sémaphores

- Exemple: *sem_posix.c*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define SEMINITVALUE 1
#define MAXTHREADS 100

sem_t my_sem;

void* critical_section(void* arg){
    // Extraire l'index du thread
    int myindex = *(int*) arg;

    // Attendre que le sémaphore soit disponible
    sem_wait(&my_sem);
    // Section critique
    for(int i = 0; i < 5; i++){
        printf("Iteration %d: Thread %d in critical section\n", i, myindex);
        sleep(1); // Simuler une tâche dans la section critique
    }
    // Quitter section critique
    sem_post(&my_sem);
    return NULL;
}

int main(int argc, char *argv[]){
    int i, n, error;
    pthread_t threadid[MAXTHREADS];

    if ((argc != 2) || ((n = atoi(argv[1])) <= 0) || ((n = atoi(argv[1])) > MAXTHREADS)) {
        printf("Usage: %s nombre de threads\n", argv[0]);
        exit(1);
    }

    // Initialiser le sémaphore
    sem_init(&my_sem, 0, SEMINITVALUE);

    // Création des threads
    for(i=0; i<n; i++){
        pthread_create(&threadid[i], NULL, critical_section, (void*)&i);
    }

    // Attendre que les threads se terminent
    for(i=0; i<n; i++){
        pthread_join(threadid[i], NULL);
    }

    // Detruire le sémaphore
    sem_destroy(&my_sem);
    return 0;
}
```

54

54


```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./sem_posix 2
Iteration 0: Thread 1 in critical section
Iteration 1: Thread 1 in critical section
Iteration 2: Thread 1 in critical section
Iteration 3: Thread 1 in critical section
Iteration 4: Thread 1 in critical section
Iteration 0: Thread 0 in critical section
Iteration 1: Thread 0 in critical section
Iteration 2: Thread 0 in critical section
Iteration 3: Thread 0 in critical section
Iteration 4: Thread 0 in critical section
```

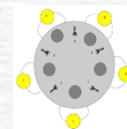
```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./sem_posix 2
Iteration 0: Thread 0 in critical section
Iteration 1: Thread 0 in critical section
Iteration 2: Thread 0 in critical section
Iteration 3: Thread 0 in critical section
Iteration 4: Thread 0 in critical section
Iteration 0: Thread 0 in critical section
Iteration 1: Thread 0 in critical section
Iteration 2: Thread 0 in critical section
Iteration 3: Thread 0 in critical section
Iteration 4: Thread 0 in critical section
```

55

55

Sémaphores

- Exemple: problème des philosophes
 - Pour cinq philosophes, seulement deux philosophes peuvent manger à un instant donné car les couverts ne sont pas suffisants.
 - Trois philosophes n'ont la possibilité que de penser ou d'être en attente de vouloir manger.
 - Quelque soit le nombre de philosophes, on ne peut jamais avoir deux philosophes mangeant cote à cote, pour de "conflit de couverts".
- L'état des philosophes sera stocké dans un tableau alloué dans un segment de mémoire partagé
- L'utilisation de ce tableau alloué en mémoire partagé, implique immédiatement l'usage d'un sémaphore d'exclusion mutuelle.



56

56

Sémaphores

- Exemple: problème des philosophes, *philosophe1.c*

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5
#define G ((N+1-1)%N) // Philosophe de gauche de i
#define D (i) // Philosophe de droite de i
#define libre 1
#define occupe 0

// 1 = libre, 0 = occupé
int fourchettes[N] = {libre, libre, libre, libre, libre};
// Sémaphore pour protéger l'accès aux fourchettes
sem_t mutex;

int main() {
    // Identifiants des philosophes
    int NumPhi[N] = {0, 1, 2, 3, 4};
    int i;
    pthread_t ph[N];
    sem_init(&mutex, 0, 1); // Initialiser le sémaphore
    // Créer les threads pour les philosophes
    for (i = 0; i < N; i++) {
        pthread_create(&ph[i], NULL, philosophe, (void*)&NumPhi[i]);
    }
    // Attendre que les threads se terminent
    i = 0;
    while(i < N){
        pthread_join(ph[i], NULL);
        i++;
    }

    sem_destroy(&mutex); // Détruire le sémaphore
    return 0;
}

// La fonction que chaque philosophe exécute
void* philosophe(void* arg) {
    int i = *(int*)arg; // L'identifiant du philosophe
    int nb = 3; // Nombre de fois que le philosophe va manger
    while (nb){
        // Penser
        printf("Philosophe %d pense\n", i);
        sleep(1); // Simuler le temps de penser
        // Tenter de prendre les fourchettes
        sem_wait(&mutex);
        if(fourchettes[G] && fourchettes[D]){
            // Prendre les fourchettes
            fourchettes[G] = occupe;
            fourchettes[D] = occupe;
            printf("Philosophe %d mange\n", i);
            sem_post(&mutex);
            nb--; // Le philosophe a mangé une fois
            sleep(1); // Simuler le temps de manger
            // Libérer les fourchettes
            sem_wait(&mutex);
            fourchettes[G] = libre;
            fourchettes[D] = libre;
            printf("Philosophe %d a fini de manger\n", i);
            sem_post(&mutex);
        } else {
            // Les fourchettes ne sont pas disponibles, le philosophe continue de penser
            printf("Philosophe %d ne peut pas manger, les fourchettes sont occupées\n", i);
            sem_post(&mutex);
        }
    }
    return NULL;
}
```

57

57

Sémaphores

- Exemple: problème des philosophes, *philosophe1.c*
- Cette solution résout le problème d'interblocage.
- Un philosophe peut mourir de faim car il ne pourra jamais obtenir les fourchettes nécessaires pour manger (problème de famine).
- Pour éviter le problème de famine il faut garantir que si un processus demande d'entrer en section critique, il obtient satisfaction au bout d'un temps fini.
- Lorsqu'un philosophe *i* ne parvient pas à prendre les fourchettes, il se met en attente ($P(S[i])$).
- Lorsqu'un philosophe termine de manger, il vérifie si ses voisins sont en attente. Si c'est le cas, il réveille les voisins qui peuvent manger en appelant l'opération V.
- On distingue trois états pour les philosophes : penser, manger et faim.

```
Philosophe 3 pense
Philosophe 0 ne peut pas manger, les fourchettes sont occupées
Philosophe 0 pense
Philosophe 1 a fini de manger
Philosophe 4 mange
Philosophe 2 mange
Philosophe 3 ne peut pas manger, les fourchettes sont occupées
Philosophe 3 pense
Philosophe 0 ne peut pas manger, les fourchettes sont occupées
Philosophe 0 pense
Philosophe 4 a fini de manger
Philosophe 2 a fini de manger
Philosophe 3 mange
Philosophe 0 mange
Philosophe 3 a fini de manger
Philosophe 3 pense
Philosophe 0 a fini de manger
Philosophe 0 pense
Philosophe 0 mange
Philosophe 3 mange
Philosophe 3 a fini de manger
Philosophe 0 a fini de manger
saa@NS4AD:~/Desktop/SystProg/Chap1$
```

58

58

Sémaphores

■ Problème des rédacteurs et des lecteurs

```
#include<stdio.h>
#include<unistd.h>
#include<pthread.h>
#include<semaphore.h>

#define N 3
#define ACCES 4

int Nbl = 0; // Nombre de lecteurs actifs
sem_t Redit;
sem_t mutex;
void* lecteur(void *);
void* redacteur(void *);

int main()
{
    int i;
    int numred[N] = {0, 1, 2};
    int numlec[N] = {0, 1, 2};
    pthread_t red[N], lec[N];
    sem_init(&Redit, 0, 1);
    sem_init(&mutex, 0, 1);

    for(i=0; i<N; i++){
        // Création des threads lecteurs et rédacteurs
        pthread_create(&lec[i], NULL, lecteur, (void*)&numlec[i]);
        pthread_create(&red[i], NULL, redacteur, (void*)&numred[i]);
    }
    // Attendre que les threads se terminent
    for(i=0; i<N; i++){
        pthread_join(lec[i], NULL);
        pthread_join(red[i], NULL);
    }
    printf("Fin des threads lecteurs et rédacteurs\n");
    sem_destroy(&Redit);
    sem_destroy(&mutex);
    return 0;
}
```

```
void *lecteur(void *arg){
    int i = *(int*)arg;
    int x = ACCES;
    do{
        printf("Lecteur %d veut lire\n", i);
        sem_wait(&mutex);
        if(!Nbl){
            // Si aucun lecteur n'est actif, bloquer les rédacteurs
            sem_wait(&Redit);
        }
        Nbl++;
        // Libérer le mutex pour permettre à d'autres lecteurs de vérifier Nbl
        sem_post(&mutex);
        // Simuler la lecture
        printf("Lecteur %d lit\n", i);
        sleep(1);
        // Fin de la lecture
        sem_wait(&mutex); // Protéger l'accès à Nbl
        Nbl--;
        if(!Nbl){
            // Si aucun lecteur n'est actif, débloquent les rédacteurs
            sem_post(&Redit);
        }
        // Libérer le mutex pour permettre à d'autres lecteurs de vérifier
        sem_post(&mutex);
    } while(--x);
    printf("Lecteur %d a fini de lire\n", i);
}

void *redacteur(void *arg){
    int i = *(int*)arg;
    int x = ACCES;
    do {
        printf("Rédacteur %d veut écrire\n", i);
        // Bloquer les rédacteurs pour garantir l'exclusion mutuelle
        sem_wait(&Redit);
        // Simuler l'écriture
        printf("Rédacteur %d écrit\n", i);
        sleep(1);
        // Libérer le sémaphore pour permettre à d'autres rédacteurs ou lecteurs d'accéder
        sem_post(&Redit);
    } while(--x);
    printf("Rédacteur %d a fini d'écrire\n", i);
}
```

62

62

Sémaphores

■ Producteur/consommateur

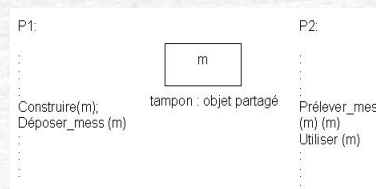
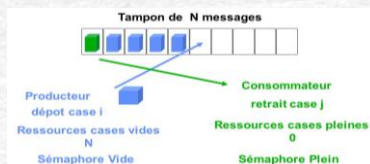
- Deux processus partagent une mémoire tampon de taille fixe. Les deux processus ne doivent pas accéder en même temps au tampon

■ Le producteur

- Met des informations dans la mémoire tampon, et l'autre, les retire
- Peut produire uniquement si le tampon n'est pas plein
- Doit être bloqué tant et aussi longtemps que le tampon est plein

■ Le consommateur

- Peut retirer un objet du tampon uniquement si le tampon n'est pas vide
- Doit être bloqué tant et aussi longtemps que le tampon est vide

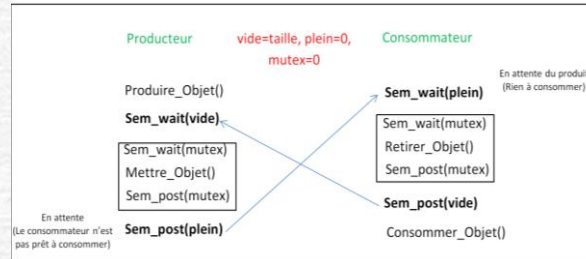


63

63

Producteur/consommateur

- Producteur/consommateur avec sémaphores utilise trois sémaphores
 - Le premier sémaphore, nommé **plein**, compte le nombre d'emplacements occupés. Il est initialisé à 0, utilisé par le consommateur
 - Le second, nommé **vide**, compte le nombre d'emplacements libres. Il est initialisé à N (la taille du tampon), utilisé par le producteur
 - Le dernier, nommé **mutex**, assure l'exclusion mutuelle pour l'accès au tampon (lorsqu'un processus utilise le tampon, les autres ne peuvent pas y accéder)
- Le producteur attend si le tampon est plein : P(vide). Il est réveillé dès que le tampon n'est plus plein: V(vide)
- Le consommateur attend si le tampon est vide : P(plein). Il est réveillé dès que le tampon n'est plus vide : V(plein)



64

64

Producteur/consommateur

- Le sémaphore vide contrôle le nombre de cases vides
- Le producteur exécute P(vide) pour réserver une case
- Le consommateur exécute V(vide) pour libérer une case
- Le producteur exécute V(plein) pour signaler au consommateur 1 nouveau message dans le tampon
- Le consommateur exécute P(plein) pour attendre un message

- Pas d'interblocage:
Le producteur et le consommateur ne peuvent être bloqués simultanément, respectivement sur P(vide) et P(plein)

```
#include<semaphore.h>
#include<unistd.h> //sleep
#include<pthread.h>
#include<stdio.h>
#define taille 3
sem_t plein , vide , mutex ;
int tampon [ taille ] ;
pthread_t cons , prod ;
void * consommateur ( ) ;
void * producteur ( ) ;

int main ( )
{
    // initialiser les sémaphores
    sem_init(&plein , 0 , 0 ) ;
    sem_init(&vide , 0 , taille ) ;
    sem_init(&mutex , 0 , 1 ) ;
    //créer les threads
    pthread_create(&cons , NULL , consommateur , NULL) ;
    pthread_create(&prod , NULL , producteur , NULL) ;
    // attendre la fin des threads
    pthread_join ( prod , NULL ) ;
    pthread_join ( cons , NULL ) ;
    printf ( " fin des thread \n" ) ;
    return 0 ;
}
```

65

65

Producteur/consommateur

Exemple: taille 1

```
void * consommateur ( )
{
    int ic = 0 , nbcons = 0 , objet ;
    do {
        sem_wait(&plein ) ;
        sem_wait(&mutex ) ;
        // consommer
        objet = tampon [ ic ] ;
        sem_post (&mutex ) ;
        sem_post (&vide ) ;
        printf ( " ici cons . : tampon[%d]= %d\n" , ic , objet ) ;
        ic =( ic+1)% taille ;
        nbcons ++;
        sleep ( 2 ) ;
    } while ( nbcons <=5 ) ;
    return (NULL) ;
}
```

```
void * producteur ( )
{
    int ip = 0 , nbprod = 0 , objet =0;
    do {
        sem_wait(&vide ) ;
        sem_wait(&mutex ) ;
        // produire
        tampon [ ip ]= objet ;
        sem_post (&mutex ) ;
        sem_post (&plein ) ;
        printf( " ici prod . : tampon[%d]= %d\n" , ip , objet ) ;
        objet ++;
        nbprod++;
        ip =( ip+1)% taille ;
    } while ( nbprod <=5 ) ;
    return (NULL);
}
```

66

66

Plan

- La notion des threads
- Communication entre threads
- **Synchronisation**
 - Verou
 - Semaphore
 - **Moniteur**

67

67

Moniteurs

- Un moniteur est une structure de variables et de procédures pouvant être paramétrée et partagée par plusieurs process.
- On dit que **l'entrée** du moniteur par un processus **exclut** l'entrée du moniteur par un autre processus.
- Au lieu d'être dispersées dans plusieurs processus, les **sections critiques** sont transformées en **procédures d'un moniteur**.
- La gestion des sections critiques n'est plus à la charge de l'utilisateur, mais elle est réalisée par le moniteur.
- Il utilise des **variables de type condition** et deux primitives agissant sur elles :
 - **WAIT** : bloque le processus appelant et autorise un processus en attente à entrer dans le moniteur,
 - **SIGNAL** : réveille le processus endormi en tête de la file d'attente.

68

68

Moniteurs

- Les moniteurs proposent une solution de «haut-niveau» pour la protection de données partagées.
- Ils simplifient la mise en place de sections critiques.
- Seul un processus (ou tâche ou thread) peut être **actif** à un moment donné à **l'intérieur du moniteur**.
- La demande d'entrée dans un moniteur (ou d'exécution d'une primitive du moniteur) sera bloquante tant qu'il y aura un processus actif à l'intérieur du moniteur.
- L'accès à un moniteur construit donc implicitement une exclusion mutuelle.

69

69

Moniteurs

- **Variables de condition:** permet à une thread de se mettre en attente d'une **condition logique** (prédicat) du programme, tout en libérant l'exclusion mutuelle imposée par le *mutex*.

```
int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t *mutex);
```

- L'activité appelante doit posséder le verrou *mutex*.
- L'activité est alors **bloquée sur la variable condition** après avoir libéré le verrou.
- L'activité reste bloquée jusqu'à ce que la **variable condition soit signalée** et que l'activité ait réussi à **réacquérir le verrou**.

70

70

Moniteurs

```
int pthread_cond_signal (pthread_cond_t *cond_wait);
```

- Signale la variable condition:
 - Une activité bloquée sur la variable condition est réveillée.
 - Cette activité tente alors de réacquérir le verrou correspondant à son appel de *cond_wait*.
 - Elle sera effectivement **débloquée** quand elle **réussira à réacquérir ce verrou**.
 - Il n'y a aucun ordre garanti pour le choix de l'activité réveillée.
 - L'opération *signal* n'a aucun effet s'il n'y a aucune activité bloquée sur la variable condition (pas de mémorisation).

```
int pthread_cond_broadcast (pthread_cond_t *cond);
```

- Toutes les activités en attente sont réveillées, et tentent d'obtenir le verrou correspondant à leur appel de *cond_wait*.

71

71

Exemple du parking:

```
#include<stdio.h>
#include<unistd.h>
#include<pthread.h>
#include<stdlib.h>

#define N 5 // Nombre de place dans le parking
#define NbTh 10 // Nombre de voitures

pthread_t tid[NbTh];
pthread_mutex_t mutex;
pthread_cond_t cond;

// Initialisation
int NbEntrant = 0; // Nombre de voitures entrantes
int NbSortant = 0; // Nombre de voitures sortantes
int NBV = 0; // Nombre de voitures dans le parking

void Demander_a_emprunter_la_passrelle(){
    pthread_mutex_lock(&mutex);
    while(((NBV + NbEntrant) >= N) || (NbSortant > 0)){
        pthread_cond_wait(&cond, &mutex);
    }
    NbEntrant++;
    pthread_mutex_unlock(&mutex);
}

void Se_garer(){
    pthread_mutex_lock(&mutex);
    NBV++;
    NbEntrant--;
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&mutex);
}

void Demander_a_sortir(){
    pthread_mutex_lock(&mutex);
    NbSortant++;
    while(NbEntrant > 0){
        pthread_cond_wait(&cond, &mutex);
    }
    NBV--;
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&mutex);
}

void Quitte_la_passrelle(){
    pthread_mutex_lock(&mutex);
    NbSortant--;
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&mutex);
}
```

Moniteurs

```
void *fnc(void *arg){
    srand(pthread_self());
    printf("La voiture %d veut emprunter la passrelle pour entrer dans le parking\n", (int)arg);
    Demander_a_emprunter_la_passrelle();

    // Temps de la traversée de la passrelle
    printf("La voiture %d emprunte la passrelle pour se garer\n", (int)arg);
    usleep(1000);

    printf("La voiture %d se gare\n", (int)arg);
    Se_garer();

    // Le temps de stationnement
    usleep(rand() % 1000000);

    printf("La voiture %d veut emprunter la passrelle pour sortir du parking\n", (int)arg);
    Demander_a_sortir();

    printf("La voiture %d sort du parking et emprunte la passrelle\n", (int)arg);
    // Temps de la traversée de la passrelle
    usleep(1000);

    Quitte_la_passrelle();
    printf("La voiture %d quitte la passrelle\n", (int)arg);
    return NULL;
}

int main(){
    int num,i;
    pthread_mutex_init(&mutex, 0);
    pthread_cond_init(&cond, 0);

    // Creation des threads
    for(i=0; i<NbTh; i++){
        num = i;
        pthread_create(&tid[i], NULL, fnc, (void*)num);
    }

    // Attente de la fin des threads
    for(i=0; i<NbTh; i++){
        pthread_join(tid[i], NULL);
    }

    return 0;

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    exit(0);
}
```

73

73



```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./parking
La voiture 0 veut emprunter la passrelle pour entrer dans le parking
La voiture 0 emprunte la passrelle pour se garer
La voiture 1 veut emprunter la passrelle pour entrer dans le parking
La voiture 1 emprunte la passrelle pour se garer
La voiture 3 veut emprunter la passrelle pour entrer dans le parking
La voiture 3 emprunte la passrelle pour se garer
La voiture 4 veut emprunter la passrelle pour entrer dans le parking
La voiture 4 emprunte la passrelle pour se garer
La voiture 5 veut emprunter la passrelle pour entrer dans le parking
La voiture 6 veut emprunter la passrelle pour entrer dans le parking
La voiture 2 veut emprunter la passrelle pour entrer dans le parking
La voiture 7 veut emprunter la passrelle pour entrer dans le parking
La voiture 5 emprunte la passrelle pour se garer
La voiture 8 veut emprunter la passrelle pour entrer dans le parking
La voiture 9 veut emprunter la passrelle pour entrer dans le parking
La voiture 0 se gare
La voiture 1 se gare
La voiture 3 se gare
La voiture 4 se gare
La voiture 5 se gare
```

```
La voiture 5 quitte la passrelle
La voiture 2 emprunte la passrelle pour se garer
La voiture 2 se gare
La voiture 8 veut emprunter la passrelle pour sortir du parking
La voiture 8 sort du parking et emprunte la passrelle
La voiture 8 quitte la passrelle
La voiture 3 veut emprunter la passrelle pour sortir du parking
La voiture 3 sort du parking et emprunte la passrelle
La voiture 3 quitte la passrelle
La voiture 0 veut emprunter la passrelle pour sortir du parking
La voiture 0 sort du parking et emprunte la passrelle
La voiture 0 quitte la passrelle
La voiture 2 veut emprunter la passrelle pour sortir du parking
La voiture 2 sort du parking et emprunte la passrelle
La voiture 2 quitte la passrelle
La voiture 6 veut emprunter la passrelle pour sortir du parking
La voiture 6 sort du parking et emprunte la passrelle
La voiture 6 quitte la passrelle
saad@NSAAD:~/Desktop/SystProg/Chap3$
```

74

74

References

- Samia Bouzefrane, Les Systèmes d'exploitation: Cours et Exercices corrigés Unix, Linux et Windows XP avec C et JAVA (566 pages), Dunod Editeur, Octobre 2003, ISBN : 2 10 007 189 0.
- Jean-François Peyre, supports de cours sur l'informatique industrielle-systèmes temps réel, CNAM(Paris).
- A. Queudet, Cours Systèmes d'exploitation, Univ. Nantes
- M. DALMAU - IUT de Bayonne - Les systèmes d'exploitation.
- <http://queinnec.perso.enseeiht.fr/Ens/SC/precis.pdf>